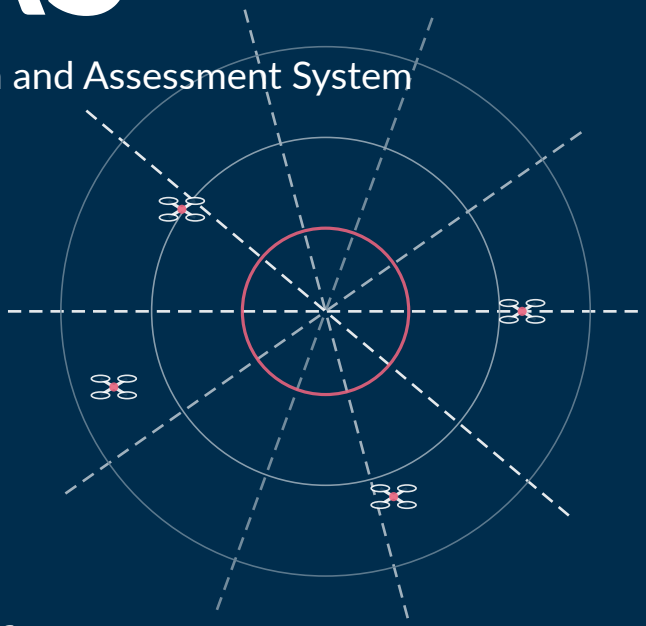
MUSAS

Modular UTM Simulation and Assessment System



User Guide & API Reference

Version 0.1.0

Caio Bianchi

Instituto Tecnológico de Aeronáutica

`github.com/caio-bianchi/musas`

Companion notebook: `documentation/getting_started.ipynb`

November 6, 2026

Contents

1 Overview **1**

1.1 Architecture . . . 1

1.2 Conventions . . . 1

1.3 Package map . . . 2

2 Installation **3**

2.1 Requirements . . . 3

2.2 Installing directly from Git . . . 3

2.3 Development install . . . 4

2.4 Checking the installation . . . 4

3 Quick Start: Your First Simulation **5**

3.1 Imports and output folders . . . 5

3.2 Airspace . . . 5

3.3 Flight plans . . . 6

3.4 Strategic algorithms . . . 6

3.5 Simulation and UASs . . . 7

3.6 Running . . . 7

3.7 Metrics . . . 8

3.8 Plotting . . . 8

4 Going Further **10**

4.1 Tactical layer and dynamic geofences . . . 10

4.2 Plugging in your own algorithm . . . 12

4.3 Reading the dataset . . . 12

4.4 Animations . . . 13

5 Simulation Guide **14**

5.1 Life cycle of an operation . . . 14

5.2 Choosing the deconfliction layers . . . 14

5.3 Timing parameters . . . 15

5.4 Processing time . . . 15

5.5 Airspace changes and disturbances . . . 15
5.6 UAS status . . . 16
5.7 Datasets . . . 16
5.8 Metrics . . . 17

6 API Reference **18**
6.1 `musas.simulation` . . . 18
6.2 `musas.simulation.metrics` . . . 23
6.3 `musas.domain` . . . 24
6.4 `musas.airspace` . . . 24
6.5 `musas.algorithms`: built-in algorithms . . . 25

7 Writing Your Own Algorithms **28**
7.1 Strategic contracts . . . 28
7.2 Tactical contracts . . . 29
7.3 Vehicle contracts . . . 29
7.4 Rules for every algorithm . . . 29
7.5 Template: a tactical algorithm . . . 30

8 Troubleshooting **31**

1

Overview

MUSAS is an open-source Python framework for simulating UAS Traffic Management (UTM) and for assessing **strategic** and **tactical deconfliction** strategies. It is built on the SimPy discrete-event engine and keeps every UTM function as an independent, replaceable algorithm. You describe the traffic, the airspace and the algorithms; MUSAS executes the operations, exchanges the messages, records everything in a SQLite dataset, and computes metrics, figures, and animations from it.

Replaceable algorithms. Tracking, conflict prediction, strategic and tactical deconfliction, guidance, dynamics, and sensors are plugged in through explicit contracts.

Non-nominal conditions. Wind zones, obstacles, geofences activated during the run, network and processing delays.

Persistent datasets. Every run is stored in SQLite: plans, trajectories, reports, tracks, airspace changes, and module events.

Assessment. Safety, operational and efficiency metrics, publication-ready plots, and animations, all rebuilt from the dataset.

1.1 Architecture

Figure 1.1 shows the logical architecture. The UASs and the UTM modules exchange messages through a simulated network. The green modules are part of the framework; the red arrows mark where your algorithms are plugged in.

- The **Strategic Deconfliction Module** receives each proposed 4D flight plan before departure and approves it, returns a counter-proposal, or rejects it.
- The **Flight Plan Manager** stores the authorized plans, activates them at departure, and gives the algorithms a snapshot of the airspace (obstacles and *active* geofences; wind is never exposed).
- The **Tracking Module** turns the periodic reports of each UAS into a predicted track.
- The **Tactical Deconfliction Module** periodically checks the tracks for conflicts and issues flight-plan updates during flight.

1.2 Conventions

- **Space.** Cartesian coordinates (x, y, z) in metres; z is the altitude.
- **Time.** Simulated seconds. A run of 60 s may take a fraction of a second of wall-clock time.

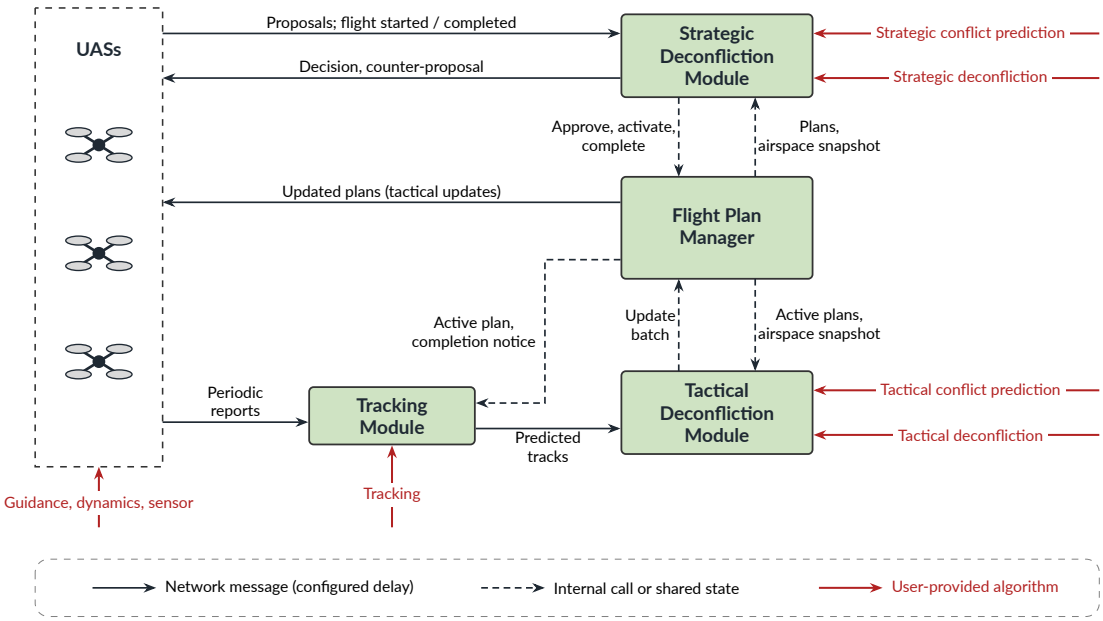


Figure 1.1: Logical architecture of MUSAS. Solid arrows are network messages, dashed arrows are internal calls, and red arrows are user-provided algorithms.

- **Flight plans.** A `FlightPlan` is a list of 4D `Waypoints` (x, y, z, t). The first waypoint is the initial position and its time is the departure time; the UAS flies straight segments and should reach each waypoint at its time.
- **Separation.** The built-in algorithms use a spherical separation: a loss of separation (LoS) occurs when two UASs are closer than the required distance.

1.3 Package map

Module	What you find there
`musas.simulation`	`Simulation`, results, datasets, plotting, animation, `calculate_metrics`
`musas.simulation.metrics`	Individual metric functions
`musas.domain`	`Waypoint`, `FlightPlan`, `FlightPlanUpdate`, `Conflict`, tracking types, enums
`musas.airspace`	`Airspace`, `Obstacle`, `Geofence`, `WindZone`, `Box3D`, `Cylinder3D`
`musas.algorithms`	Algorithm contracts (also in `.interfaces`) and built-in implementations (also in `.builtins`)
`musas.uav`	`UAVStatus` and the internal UAV model

2

Installation

2.1 Requirements

- Python **3.11 or newer** and `pip`.
- Git, to install directly from GitHub.
- Dependencies installed automatically: `simpy`, `numpy`, `matplotlib`, `pillow`.
- Optional: `jupyter` for the companion notebook and `ffmpeg` for MP4 animations (GIF needs nothing else).

2.2 Installing directly from Git

The package does not need to be cloned or downloaded: `pip` can install it straight from the GitHub repository. We recommend a virtual environment:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install "git+https://github.com/caio-bianchi/musas.git"
```

Then import the package as `musas`:

```
import musas
from musas.simulation import Simulation
print(musas.__version__)       # 0.1.0
```

> **Note.** Use `pip show musas` to inspect the installed version and `pip uninstall musas` to remove it.

2.2.1 Choosing a version

Append `@<ref>` to the URL to install a specific branch, tag, or commit:

```
pip install "git+https://github.com/caio-bianchi/musas.git@main"      # branch
pip install "git+https://github.com/caio-bianchi/musas.git@v0.1.0"    # tag
pip install "git+https://github.com/caio-bianchi/musas.git@ddbce49"   # commit
```

To get the latest commit again after an update of the repository:

```
pip install --upgrade --force-reinstall --no-deps \
    "git+https://github.com/caio-bianchi/musas.git"
```

2.2.2 Other ways to declare the dependency

```
# Jupyter or Google Colab (restart the kernel afterwards)
%pip install "git+https://github.com/caio-bianchi/musas.git"

# requirements.txt
musas @ git+https://github.com/caio-bianchi/musas.git

# pyproject.toml
dependencies = ["musas @ git+https://github.com/caio-bianchi/musas.git"]

# SSH instead of HTTPS (e.g. private forks)
pip install "git+ssh://git@github.com/caio-bianchi/musas.git"
```

2.3 Development install

To modify MUSAS itself, clone the repository and install it in *editable* mode; changes to the source are visible without reinstalling:

```
git clone https://github.com/caio-bianchi/musas.git
cd musas
pip install -e .
```

2.4 Checking the installation

```
python -c "import musas, musas.simulation; print('MUSAS', musas.__version__)"
```

> **Tip.** The companion notebook `documentation/getting_started.ipynb` contains every listing of Chapters 3 and 4. Install Jupyter (`pip install jupyter`), open the `documentation` folder, and run all cells.

3

Quick Start: Your First Simulation

In this chapter you will build, run, and assess a complete scenario in eight short steps. Two UASs, A and B, would reach the origin at the same time, and the route of B crosses a communication tower. The strategic layer must solve both problems before departure.

Each listing is a cell of `getting_started.ipynb`, in the same order; the label at the top right of each listing names the corresponding file.

3.1 Imports and output folders

notebook: 01_setup.py

```
from pathlib import Path

from musas.airspace import Airspace, Cylinder3D, Obstacle
from musas.algorithms import (
    DetourPathPlanner,
    FirstOrderDynamics,
    MinimumSeparationConflictPrediction,
    PerfectSensor,
    RecursiveTimeShiftDeconfliction,
    WaypointGuidance,
)
from musas.domain import FlightPlan, Waypoint
from musas.simulation import (
    Simulation,
    SimulationDataset,
    VisualizationConfig,
    calculate_metrics,
    plot_dataset,
)

OUTPUTS = Path("outputs")      # SQLite datasets written by the runs
FIGURES = Path("figures")      # figures used in the PDF documentation
OUTPUTS.mkdir(exist_ok=True)
FIGURES.mkdir(exist_ok=True)
```

3.2 Airspace

An `Airspace` holds the permanent obstacles and the geofences and wind zones that exist from the start. Here, a vertical cylinder of radius 15 m at $(0, 50)$ represents a tower.

notebook: 02_airspace.py

```
airspace = Airspace()
airspace.add_obstacle(
    Obstacle(
        obstacle_id="tower",
        name="Communication tower",
        geometry=Cylinder3D(
            center_x=0.0, center_y=50.0, radius=15.0,
            minimum_z=0.0, maximum_z=150.0,
        ),
    )
)
```

3.3 Flight plans

Each UAS proposes a 4D plan. The first waypoint gives the initial position and the departure time; here both UASs depart at $t = 2$ s and fly 200 m in 20 s (10 m/s).

notebook: 03_flight_plans.py

```
# Waypoint(x, y, z, t): position in metres, time in seconds.
flight_plans = {
    "UAV A": FlightPlan([
        Waypoint(-100.0, 0.0, 100.0, 2.0),
        Waypoint(100.0, 0.0, 100.0, 22.0),
    ]),
    "UAV B": FlightPlan([
        Waypoint(0.0, -100.0, 100.0, 2.0),
        Waypoint(0.0, 100.0, 100.0, 22.0),
    ]),
}
```

3.4 Strategic algorithms

The strategic layer needs two algorithms: a *conflict predictor* and a *deconfliction* algorithm. The built-in predictor samples the plans and reports a traffic conflict when they come closer than 30 m, or an airspace conflict when a sample enters an obstacle or active geofence. The built-in deconfliction removes airspace conflicts with a lateral detour and traffic conflicts by delaying the departure in steps of 3 s.

notebook: 04_algorithms.py

```
strategic_predictor = MinimumSeparationConflictPrediction(
    minimum_separation_distance=30.0,
)
strategic_deconfliction = RecursiveTimeShiftDeconfliction(
    conflict_prediction_algorithm=strategic_predictor,
    path_planner=DetourPathPlanner(lateral_clearance=15.0),
    time_shift_step=3.0,
)
```

3.5 Simulation and UASs

The `Simulation` wires everything together. Each UAS receives its own dynamics, guidance and sensor models.

notebook: 05_simulation.py

```
QUICKSTART_DATASET = OUTPUTS / "quickstart.sqlite"

simulation = Simulation(
    airspace=airspace,
    strategic_conflict_prediction_algorithm=strategic_predictor,
    strategic_deconfliction_algorithm=strategic_deconfliction,
    dataset_path=QUICKSTART_DATASET,
    overwrite_dataset=True,
)

for name, flight_plan in flight_plans.items():
    simulation.add_uav(
        name=name,
        flight_plan=flight_plan,
        dynamics_algorithm=FirstOrderDynamics(
            max_acceleration=10.0, velocity_time_constant=0.3,
        ),
        guidance_algorithm=WaypointGuidance(max_speed=15.0),
        sensor_model=PerfectSensor(),
        waypoint_acceptance_radius=2.0,
    )
```

Note. Omit `dataset_path` to keep the results only in memory; the returned `SimulationResult` can still be saved later with `result.save(path)`.

3.6 Running

`run(until)` advances the simulation to an absolute simulated time. `close()` flushes and closes the dataset.

notebook: 06_run.py

```
result = simulation.run(until=60.0)
simulation.close()      # flush and close the SQLite dataset

for name, uav in result.uavs.items():
    plan = result.latest_flight_plans[name]
    print(f"{name}: {uav.status.name}, departs at "
          f"t = {plan.waypoint_list[0].t:.1f} s, "
          f"{len(plan.waypoint_list)} waypoints")
```

output

```
UAV A: COMPLETED, departs at t = 2.0 s, 2 waypoints
UAV B: COMPLETED, departs at t = 8.0 s, 4 waypoints
```

UAV A was approved unchanged. UAV B received a counter-proposal: a detour around the tower (two extra waypoints) and a departure delayed from 2 to 8 s, after two time shifts.

3.7 Metrics

Metrics are computed after the run, from the dataset:

notebook: 07_metrics.py

```python
with SimulationDataset(QUICKSTART_DATASET) as dataset:
    metrics = calculate_metrics(
        dataset,
        minimum_separation_distance=30.0,
        collision_distance=1.0,
    )

for key in ("flights", "modified_flights", "detected_strategic_conflicts",
            "loss_of_separation_events", "collision_events",
            "minimum_pairwise_separation", "average_actual_distance"):
    value = metrics[key]
    print(f"{key:30s} {value:.1f}" if isinstance(value, float)
          else f"{key:30s} {value}")
```

output

```
flights                        2
modified_flights               1
detected_strategic_conflicts   2
loss_of_separation_events      0
collision_events               0
minimum_pairwise_separation    30.5
average_actual_distance        209.3
```

3.8 Plotting

notebook: 08_plot.py

```python
figure = plot_dataset(
    QUICKSTART_DATASET,
    config=VisualizationConfig(view="xy", show_flight_plans=True,
                               spatial_unit="m"),
    timestamp=14.0,
    output_path=FIGURES / "quickstart_xy.pdf",
)
```

> **Tip.** `view` accepts `"3d"`, `"xy"`, `"xz"` and `"yz"`. Saving to `.pdf` or `.svg` produces vector figures.

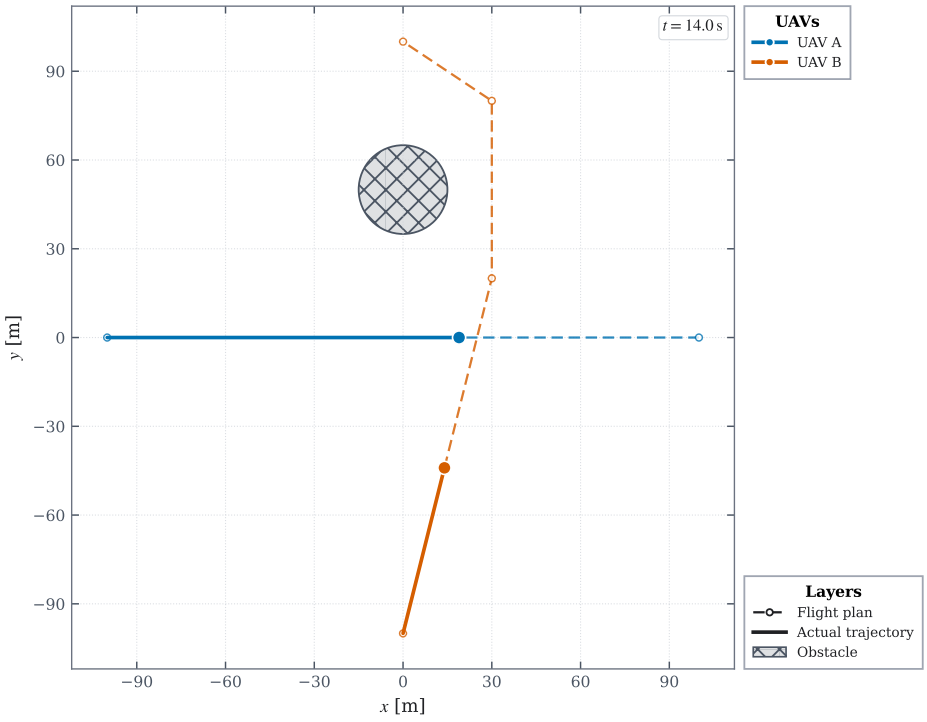


Figure 3.1: Quick-start scenario at $t = 14$ s (xy view). Dashed lines are the current flight plans and solid lines the executed trajectories. UAV B departed later and detours around the tower.

4

Going Further

4.1 Tactical layer and dynamic geofences

The tactical layer acts during flight. It needs three algorithms: a *tracking* algorithm, which predicts each UAS from its reports, a *tactical conflict predictor*, and a *tactical deconfliction* algorithm. In this new scenario a no-fly zone appears 40 m ahead of UAV C while it is flying, so only the tactical layer can react. The strategic layer is disabled by omitting both strategic algorithms.

notebook: 09_tactical.py

```
from musas.airspace import Geofence
from musas.algorithms import (
    FlightPlanAwareTracking,
    HybridGeometricManeuver,
    MinimumSeparationTacticalConflictPrediction,
)

TACTICAL_DATASET = OUTPUTS / "tactical.sqlite"

tactical_predictor = MinimumSeparationTacticalConflictPrediction(
    minimum_separation_distance=30.0, maximum_time_difference=0.25,
)
tactical_simulation = Simulation(
    airspace=Airspace(),
    # no strategic layer: both strategic algorithms are omitted
    tracking_algorithm=FlightPlanAwareTracking(),
    tactical_conflict_prediction_algorithm=tactical_predictor,
    tactical_deconfliction_algorithm=HybridGeometricManeuver(
        conflict_prediction_algorithm=tactical_predictor,
        minimum_separation_distance=30.0,
        airspace_clearance=15.0,
        maneuver_duration=4.0,
        maximum_speed=15.0,
        cooldown=2.0,
    ),
    report_interval=0.25,
    prediction_horizon=24,          # 24 states x 0.25 s = 6 s ahead
    prediction_step=0.25,
    tactical_assessment_interval=0.25,
    dataset_path=TACTICAL_DATASET,
    overwrite_dataset=True,
)

# A no-fly zone appears 40 m ahead of the UAV while it is flying.
tactical_simulation.schedule_geofence(
    Geofence(
        geofence_id="temporary-area",
        geometry=Cylinder3D(20.0, 0.0, 20.0, 0.0, 200.0),
```

```
    ),
    activation_time=10.0,
)
tactical_simulation.add_uav(
    name="UAV C",
    flight_plan=FlightPlan([
        Waypoint(-120.0, 0.0, 100.0, 2.0),
        Waypoint(120.0, 0.0, 100.0, 26.0),
    ]),
    dynamics_algorithm=FirstOrderDynamics(
        max_acceleration=100.0, velocity_time_constant=0.0,
    ),
    guidance_algorithm=WaypointGuidance(max_speed=15.0),
    waypoint_acceptance_radius=2.0,
)
tactical_result = tactical_simulation.run(until=45.0)
tactical_simulation.close()
```

notebook: 10_tactical_plot.py

```
with SimulationDataset(TACTICAL_DATASET) as dataset:
    for event in dataset.get_events():
        if event.event in ("GEOFENCE_ACTIVATED", "TACTICAL_UPDATES_ISSUED"):
            print(f"t = {event.timestamp:5.2f} s  {event.event}")
    tactical_metrics = calculate_metrics(dataset, 30.0, 1.0)
print("geofence violations:", tactical_metrics["geofence_violations"])

figure = plot_dataset(
    TACTICAL_DATASET,
    config=VisualizationConfig(view="xy", show_flight_plans=True,
                               show_tracks=True, spatial_unit="m"),
    timestamp=12.0,
    output_path=FIGURES / "tactical_xy.pdf",
)
```

output

```
t = 10.00 s  GEOFENCE_ACTIVATED
t = 10.00 s  TACTICAL_UPDATES_ISSUED
geofence violations: 0
```

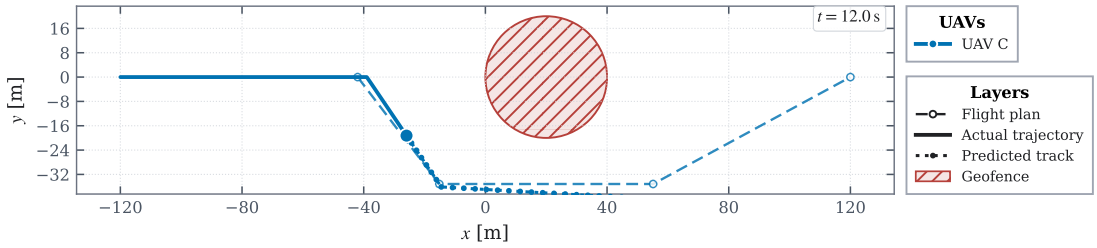


Figure 4.1: Tactical detour around a geofence activated at $t = 10$ s, shown at $t = 12$ s. The dotted line is the predicted track.

Note. The tactical layer needs `tracking_algorithm` and both tactical algorithms. The prediction horizon is `prediction_horizon` × `prediction_step` seconds; a conflict can only be detected inside it.

4.2 Plugging in your own algorithm

Any class that inherits a contract and implements its abstract method can replace a built-in algorithm. The strategic deconfliction contract receives the proposal and its predicted conflicts and returns a decision, an optional counter-proposal, and optional updates to other (inactive) plans.

notebook: 11_custom_algorithm.py

```
from musas.algorithms import StrategicDeconflictionAlgorithm
from musas.domain import StrategicDecision


class RejectOnConflict(StrategicDeconflictionAlgorithm):
    """Approve conflict-free proposals and reject all the others."""

    def deconflict(self, proposed_flight_plan, conflicts,
                   inactive_flight_plans, active_flight_plans, airspace):
        if conflicts:
            return StrategicDecision.REJECT, None, []
        return StrategicDecision.APPROVE, None, []


custom_simulation = Simulation(
    airspace=Airspace(),
    strategic_conflict_prediction_algorithm=strategic_predictor,
    strategic_deconfliction_algorithm=RejectOnConflict(),
)
for name, flight_plan in flight_plans.items():
    custom_simulation.add_uav(
        name=name,
        flight_plan=flight_plan,
        dynamics_algorithm=FirstOrderDynamics(),
        guidance_algorithm=WaypointGuidance(max_speed=15.0),
    )
custom_result = custom_simulation.run(until=40.0)
print({name: uav.status.name for name, uav in custom_result.uavs.items()})
```

output

```
{'UAV A': 'COMPLETED', 'UAV B': 'FLIGHT_PLAN_REJECTED'}
```

Chapter 7 lists every contract and the rules each one must follow.

4.3 Reading the dataset

`SimulationDataset` gives read-only access to a recorded run. Use it as a context manager so that the SQLite connection is closed.

notebook: 12_dataset.py

```
with SimulationDataset(QUICKSTART_DATASET) as dataset:
    print("UAVs:", dataset.uav_names)
    print("Run status:", dataset.run_status)

    # Strategic decisions taken for each proposal
    for event in dataset.get_events("STRATEGIC_DECISION_MADE"):
        print(f"t = {event.timestamp:.1f} s", event.details["summary"])
```

```
    # Recorded true states of one UAV
    samples = dataset.get_trajectory("UAV B", start_time=10.0, end_time=11.0)
    for sample in samples:
        print(f"t = {sample.timestamp:.1f} s",
              [round(value, 1) for value in sample.position])
```

output

```
UAVs: ('UAV A', 'UAV B')
Run status: COMPLETED
t = 0.0 s Strategic decision for UAV A: APPROVE
t = 0.0 s Strategic decision for UAV B: COUNTER_PROPOSAL
t = 10.2 s [4.3, -82.8, 100.0]
t = 10.4 s [4.8, -80.8, 100.0]
t = 10.6 s [5.3, -78.8, 100.0]
t = 10.8 s [5.8, -76.7, 100.0]
t = 11.0 s [6.3, -74.7, 100.0]
```

4.4 Animations

notebook: 13_animation.py

```
from musas.simulation import animate_dataset

animation = animate_dataset(
    QUICKSTART_DATASET,
    config=VisualizationConfig(view="xy", show_flight_plans=True,
                               frame_step=0.5, spatial_unit="m"),
    output_path=OUTPUTS / "quickstart.gif",
    fps=10,
)
```

`frame_step` is the simulated time between frames and `fps` the playback speed. Use a `.mp4` output path for video (requires `ffmpeg`).

5

Simulation Guide

5.1 Life cycle of an operation

1. At `submission_time`, the UAS sends its proposed plan to the Strategic Deconfliction Module.
2. The strategic predictor compares the proposal with the approved plans and the airspace snapshot; the strategic algorithm decides: `APPROVE`, `COUNTER_PROPOSAL`, or `REJECT`.
3. An authorized UAS waits for the time of its first waypoint and departs; its plan becomes *active*. Strategic updates may change only inactive plans.
4. In flight, the UAS sends a report every `report_interval`; the Tracking Module produces a track for each report.
5. Every `tactical_assessment_interval`, the tactical predictor checks all current tracks; the tactical algorithm may issue `FlightPlanUpdates`, which the Flight Plan Manager sends to the UASs.
6. The operation ends when the UAS reaches its last waypoint (`COMPLETED`) or collides (`COLLIDED`).

5.2 Choosing the deconfliction layers

Configuration	Arguments of `Simulation`
No strategic layer	Omit both strategic algorithms: every proposal is approved unchanged.
Strategic layer	`strategic_conflict_prediction_algorithm` and `strategic_deconfliction_algorithm` (always together).
Tracking only	`tracking_algorithm` without tactical algorithms: tracks are computed and recorded, but nothing reacts to them.
Tactical layer	`tracking_algorithm` plus both tactical algorithms.

5.3 Timing parameters

Parameter	Default	Meaning
`flight_update_interval`	0.1 s	Integration step of the vehicles and of the collision monitor
`trajectory_record_interval`	0.2 s	Sampling of the recorded true states
`report_interval`	0.5 s	Period of the UAS reports
`history_window`	6	Reports kept per UAS by the tracker
`prediction_horizon`	10	Number of predicted states per track
`prediction_step`	0.5 s	Time between predicted states
`tactical_assessment_interval`	1.0 s	Period of the tactical assessments
`network_delay`	0 s	Delivery delay of every message
`collision_distance`	1 m	Distance below which two UASs collide
`dataset_flush_interval`	10 s	Simulated time between dataset writes

5.4 Processing time

The arguments `strategic_processing_time`, `tracking_processing_time`, and `tactical_processing_time` accept a `ProcessingTimeModel`. Its `calculate(**context)` returns the simulated duration of each operation, so slow algorithms produce late decisions. The context is a dictionary with, among others, `module`, `operation` (`"conflict_prediction"`, `"deconfliction"`, or `"tracking_prediction"`), `queue_length`, and counts such as `approved_flight_plan_count`, `track_count`, or `conflict_count`.

```
from musas.simulation import ProcessingTimeModel

class LinearDelay(ProcessingTimeModel):
    """0.05 s per track plus 0.2 s per conflict."""
    def calculate(self, **context):
        return (0.05 * context.get("track_count", 0)
                + 0.2 * context.get("conflict_count", 0))
```

5.5 Airspace changes and disturbances

- Put permanent obstacles, and geofences or wind zones that exist from the start, in the initial `Airspace`.
- Use `schedule_geofence` and `schedule_wind_zone` to activate (and optionally remove) them at given times.
- `add_*` and `remove_*` change the airspace at the current simulated time, for example between two calls of `run`.
- Geofences are visible to the algorithms only after their activation. **Wind is never given to the algorithms**: it only moves the vehicles, and its effect must be inferred from the reports.

```python
from musas.airspace import Box3D, WindZone

simulation.schedule_wind_zone(
    WindZone(wind_id="gust", velocity=(0.0, 4.0, 0.0),
             geometry=Box3D(-50, 50, -200, 200, 0, 150)),
    activation_time=20.0, removal_time=40.0,
)
```

5.6 UAS status

`UAVStatus`	Meaning
`INACTIVE`	Registered, proposal not sent yet
`WAITING_FLIGHT_PLAN`	Proposal sent, waiting for the decision
`WAITING_DEPARTURE`	Authorized, waiting for the departure time
`ACTIVE`	Flying
`COMPLETED`	Reached the last waypoint and left the airspace
`FLIGHT_PLAN_REJECTED`	Proposal rejected by the strategic layer
`COLLIDED`	Collided with another UAS or an obstacle

5.7 Datasets

When `dataset_path` is given, the run is written incrementally to a SQLite file every `dataset_flush_interval`; call `close()` at the end. With `keep_history_in_memory=False`, the histories are kept only on disk, which reduces memory use in long batches. The dataset contains:

Table	Content
`metadata`	Configuration, user metadata, run status
`uavs`	Final status and model names of each UAS
`flight_plan_waypoints`	Proposed and authorized plan versions
`trajectory`	Sampled true states
`tracking_reports`, `tracking_predictions`	Reports and tracks
`airspace_objects`, `airspace_snapshots`	Obstacles, geofences, wind zones, and when each one was active
`events`	Structured events of the modules (see below)

The most useful events for analysis are listed below. Filter them by name with `get_events` and read their `details`.

Event	Recorded when
`STRATEGIC_DECISION_MADE`	A proposal is approved, counter-proposed, or rejected
`FLIGHT_PLAN_ACTIVATED`, `FLIGHT_PLAN_COMPLETED`	A flight starts or ends
`TACTICAL_CONFLICTS_DETECTED`	A tactical assessment finds conflicts
`TACTICAL_UPDATES_ISSUED`	The tactical layer issues updates
`GEOFENCE_ACTIVATED`, `GEOFENCE_REMOVED`	Dynamic geofences
`WIND_ZONE_ACTIVATED`, `WIND_ZONE_REMOVED`	Dynamic wind
`UAV_COLLIDED`	A UAS collides

5.8 Metrics

`calculate_metrics(dataset, minimum_separation_distance, collision_distance)` returns a dictionary with:

Key	Meaning
`loss_of_separation_events`	Entries of a pair of active UASs into the separation sphere
`collision_events`	Entries into the collision distance
`minimum_pairwise_separation`	Smallest recorded distance between UASs [m]
`geofence_violations`, `obstacle_violations`	Entries into active geofences / obstacles
`detected_strategic_conflicts`, `detected_tactical_conflicts`	Conflicts reported by the predictors
`flights`, `modified_flights`, `rejected_flights`	Operations, plans changed at least once, rejections
`tactical_interventions`	Tactical cycles that issued updates
`average_planned_distance`, `average_actual_distance`	Mean planned / executed distance [m]
`average_planned_time`, `average_actual_time`	Mean planned / executed flight time [s]
`per_uav`	Per-UAS planned and actual distance and time, and number of plan updates

Note. Executed distance and time consider only completed flights, so a rejected or collided operation does not appear as a shorter flight.

6

API Reference

This chapter documents every class and function a user needs. Types follow the Python annotations; `None` defaults mean “optional”.

6.1 `musas.simulation`

`Simulation` class

Assembles and coordinates one UTM simulation.

```
Simulation(airspace, strategic_conflict_prediction_algorithm=None, strategic_deconfliction_algorithm=None, tracking_algorithm=None, tactical_conflict_prediction_algorithm=None, tactical_deconfliction_algorithm=None, network_delay=0.0, …)
```

PARAMETERS

`airspace` `Airspace`
Initial airspace: obstacles and the geofences and wind zones present from the start.

`strategic_conflict_prediction_algorithm` `ConflictPredictionAlgorithm | None`
Strategic predictor; provide it together with the strategic algorithm, or omit both to disable the strategic layer.

`strategic_deconfliction_algorithm` `StrategicDeconflictionAlgorithm | None`
Strategic decision algorithm.

`tracking_algorithm` `TrackingAlgorithm | None`
Track predictor; required by the tactical layer.

`tactical_conflict_prediction_algorithm` `TacticalConflictPredictionAlgorithm | None`
Tactical predictor; provide it together with the tactical algorithm.

`tactical_deconfliction_algorithm` `TacticalDeconflictionAlgorithm | None`
Tactical decision algorithm.

`network_delay` `float = 0.0`
Delivery delay of every message [s].

`message_types_to_log` `Collection[MessageType] | None`
Message types recorded as events; by default proposals, strategic responses, flight start/-completion, and plan updates. Use `()` to log none.

`flight_update_interval` `float = 0.1`
Integration step [s].

`trajectory_record_interval` `float = 0.2`
Recording step of the true states [s].

report_interval `float = 0.5`
Period of the UAS reports [s].

history_window `int = 6`
Reports kept per UAS by the Tracking Module.

prediction_horizon `int = 10`
Number of predicted states per track.

prediction_step `float = 0.5`
Time between predicted states [s].

tactical_assessment_interval `float = 1.0`
Period of the tactical assessments [s].

collision_distance `float = 1.0`
Collision distance between UASs [m].

strategic_processing_time, tracking_processing_time, tactical_processing_time
`ProcessingTimeModel`
Simulated processing time of each module; zero by default.

dataset_path `str | Path | None`
SQLite file written during the run.

dataset_flush_interval `float = 10.0`
Simulated time between incremental writes [s].

dataset_metadata `dict | None`
Free metadata stored with the run (requires `dataset_path`).

overwrite_dataset `bool = False`
Replace an existing file.

keep_history_in_memory `bool = True`
Also keep histories in the returned result; `False` requires `dataset_path`.

Simulation.add_uav — method

`add_uav(name, flight_plan, dynamics_algorithm, guidance_algorithm, sensor_model=None, submission_time=0.0, waypoint_acceptance_radius=1.0) → UAV`
Registers a UAS and its proposed plan. Must be called before the first `run`.

PARAMETERS

name `str`
Unique identifier.

flight_plan `FlightPlan`
Proposed plan; its first waypoint is the initial position.

dynamics_algorithm `DynamicsAlgorithm`
Vehicle response to the velocity command.

guidance_algorithm `GuidanceAlgorithm`
Velocity command toward the target waypoint.

sensor_model `SensorModel | None`
Measurement model of the reports; perfect if omitted.

submission_time `float = 0.0`
When the proposal is sent [s].

waypoint_acceptance_radius `float = 1.0`
Distance at which a waypoint is considered reached [m].

Simulation.run — method

`run(until: float) → SimulationResult`
Runs or continues the simulation up to the absolute time `until`. It can be called several times with increasing times.

Simulation.close / result — methods

`close() → None`
Flushes and closes the dataset; the run status becomes `COMPLETED` if every flight ended, or `STOPPED` otherwise.
`result() → SimulationResult`
Snapshot of the state and histories collected so far.

Simulation airspace methods — methods

`schedule_geofence(geofence, activation_time, removal_time=None)`
`schedule_wind_zone(wind_zone, activation_time, removal_time=None)`
`add_geofence(geofence)  remove_geofence(geofence_id) → Geofence`
`add_wind_zone(wind_zone)  remove_wind_zone(wind_id) → WindZone`
`schedule_*` activates (and optionally removes) an object at given simulated times; `add_*`/`remove_*` act at the current time. All changes are recorded in the dataset.

SimulationResult — dataclass

Data collected up to the current time, returned by `run`.

ATTRIBUTES

time `float`
Current simulated time.

airspace `Airspace`
Airspace at the end of the run.

uavs `dict[str, UAVSnapshot]`
Final `name`, `status`, `position`, `velocity`, and model names of each UAS.

proposed_flight_plans `dict[str, FlightPlan]`
Original proposals.

flight_plan_history `dict[str, tuple[FlightPlan, ...]]`
Authorized plan versions of each UAS.

latest_flight_plans `dict[str, FlightPlan]`
(property) Latest strategic or tactical plan of each UAS.

state_history `dict[str, list[UAVStateSample]]`
Recorded `timestamp`, `position`, `velocity`, `status`.

airspace_history `list[AirspaceSnapshot]`
Geofences and wind zones present after each change.

tracking_reports, tracking_predictions `list`
Reports and tracks.

event_history `list[SimulationEvent]`
Module events.

configuration `dict`
Parameters of the run.

dataset_path `Path | None`
Dataset file, if any.

`save(path, metadata=None, overwrite=False) → Path`
Writes the result to a self-contained SQLite dataset.

SimulationDataset class

Read-only access to a recorded run. Supports `with`.
`SimulationDataset(path)` `SimulationDataset.open(path)`

PROPERTIES

metadata `dict`
Configuration and user metadata.

run_status `str`
`RUNNING`, `COMPLETED`, `STOPPED`, or `FAILED`.

simulation_time, last_committed_time `float`
Final and last written simulated times.

uav_names `tuple[str, ...]`
Registered UASs.

METHODS

get_uavs() → `list[UAVRecord]`
Final status and model names.

get_trajectory(uav_name=None, start_time=None, end_time=None) → `list[TrajectoryRecord]`
Recorded true states, optionally filtered.

get_flight_plan(uav_name, plan_kind="proposed", plan_version=0) → `FlightPlan | None`
A stored plan; `plan_kind` is `"proposed"` or `"authorized"`.

get_flight_plan_versions(uav_name, plan_kind="authorized") → `tuple[int, ...]`
Available versions.

get_events(event_type=None, component=None, start_time=None, end_time=None) → `list[StoredEvent]`
Events with `timestamp`, `component`, `event`, `details`.

get_airspace_objects(object_type=None) → `list[AirspaceObjectRecord]`
`"obstacle"`, `"geofence"`, or `"wind_zone"`.

`get_tracking_reports(uav_name=None)`
Stored reports.

`get_tracking_predictions(uav_name=None)`
Stored track states.

`get_active_geofence_ids(timestamp), get_active_wind_zone_ids(timestamp)` → `tuple[str, ...]`
Objects active at an instant.

`get_airspace_snapshots()`
When the airspace changed.

`table_count(table)` → `int`
Number of rows of a table.

`close()`
Closes the connection.

`VisualizationConfig` dataclass

Selects what `plot_dataset` and `animate_dataset` draw.

FIELDS

`view` `"3d" | "xy" | "xz" | "yz" = "3d"`
Projection.

`uav_names` `tuple[str, ...] | None`
Subset of UASs; all if `None`.

`show_real_trajectories, show_current_positions` `bool = True`
Executed trajectories and current positions.

`show_flight_plans, show_tracks, show_sensor_reports` `bool = False`
Current plans, predicted tracks, reports.

`show_geofences, show_obstacles` `bool = True`
Airspace restrictions.

`show_wind_zones` `bool = False`
Wind regions.

`show_labels, show_legend, show_grid, show_timestamp` `bool`
Labels next to the UASs (off), legend, grid, and time stamp (on).

`start_time, end_time` `float | None`
Time window of an animation.

`frame_step` `float = 1.0`
Simulated time between animation frames [s].

`figure_size` `tuple = (8.0, 5.5)`
Figure size in inches.

`title` `str | None`
Figure title.

`spatial_unit, time_unit` `str`
Axis and time-stamp units, e.g. `"m"`, `"s"`.

plot_dataset function

`plot_dataset(dataset_path, config=None, timestamp=None, output_path=None, dpi=300) → Figure`
Draws the scenario at `timestamp` (the last recorded time if omitted) and optionally saves it; `.pdf`/`.svg` produce vector figures.

animate_dataset function

`animate_dataset(dataset_path, config=None, output_path=None, fps=10.0, dpi=120, repeat=True) → FuncAnimation`
Builds an animation of the run; `output_path` must end in `.gif` or `.mp4`.

6.2 musas.simulation.metrics

calculate_metrics function

`calculate_metrics(dataset, minimum_separation_distance, collision_distance) → dict`
Computes the standard metrics of a `SimulationDataset`; the keys are listed in Section 5.8.

Each metric is also available as a function of a `SimulationDataset`:

Function	Returns
`number_of_loss_of_separation_events(ds, d)`	LoS events for separation `d`
`number_of_collision_events(ds, d)`	Collision events
`minimum_pairwise_separation(ds)`	Smallest distance [m]
`number_of_geofence_violations(ds)`	Geofence entries
`number_of_obstacle_violations(ds)`	Obstacle entries
`number_of_detected_strategic_conflicts(ds)`	Strategic conflicts
`number_of_detected_tactical_conflicts(ds)`	Tactical conflicts
`number_of_flights(ds)`	Operations
`number_of_rejected_flights(ds)`	Rejected proposals
`number_of_modified_flights(ds)`	Plans changed at least once
`number_of_tactical_interventions(ds)`	Tactical cycles with updates
`number_of_plan_updates(ds, uav)`	Plan changes of one UAS
`planned_distance(ds, uav, plan_kind, plan_version)`	Length of a stored plan [m]
`actual_distance(ds, uav)`, `actual_flight_time(ds, uav)`	Executed distance [m] / time [s] (completed flights)
`planned_flight_time(ds, uav)`	Planned duration [s]
`average_planned_distance(ds)`, `average_actual_distance(ds)`	Mean distances [m]
`average_planned_flight_time(ds)`, `average_actual_flight_time(ds)`	Mean durations [s]

6.3 musas.domain

Waypoint dataclass

`Waypoint(x: float, y: float, z: float, t: float)`
A 4D point: position [m] and the time at which it should be reached [s]. `position()` returns `[x, y, z]`; `timestamp()` returns `t`.

FlightPlan dataclass

`FlightPlan(waypoint_list: list[Waypoint] = [], priority: int = 0)`
Time-ordered trajectory; the waypoints are kept sorted by time. `add_waypoint(waypoint)` inserts a waypoint. Larger `priority` values have precedence, and every update must preserve the priority of the operation.

FlightPlanUpdate dataclass

`FlightPlanUpdate(uav_name: str, flight_plan: FlightPlan)`
A complete replacement plan for one UAS, returned by deconfliction algorithms.

Conflict base class

Empty base class of every conflict. Predictors return instances of its subclasses; define your own with a dataclass that inherits `Conflict`. The built-in subclasses are listed in Section 6.5.4.

TrackingReport, PredictedState, TrackPrediction dataclasses

`TrackingReport(uav_name, position, velocity, timestamp, received_at)`
`PredictedState(position, velocity, timestamp)`
`TrackPrediction(uav_name, estimated_state, predicted_states, generated_at)`
A report is the measured state sent by a UAS; a track is the current estimate followed by the predicted states.

StrategicDecision, MessageType enums

`StrategicDecision`: `APPROVE`, `COUNTER_PROPOSAL`, `REJECT`.
`MessageType`: `PERIODIC_REPORT`, `PROPOSAL`, `SD_RESPONSE`, `FLIGHT_STARTED`, `FLIGHT_COMPLETED`, `UPDATE_FLIGHT_PLAN`, `TRACK_UPDATE`.

6.4 musas.airspace

Airspace dataclass

`Airspace(obstacles=, geofences=, wind_zones=)`
Container of the airspace objects, indexed by identifier.

METHODS

add_obstacle(obstacle), remove_obstacle(obstacle_id)
Permanent obstacles.

add_geofence(geofence), remove_geofence(geofence_id)
Geofences.

`add_wind_zone(wind_zone)`, `remove_wind_zone(wind_id)`
Wind zones.

`active_geofences_at(t)` → `list[Geofence]`
Geofences active at `t`.

`wind_at(position, t)` → `list[float]`
Resulting wind vector.

`is_prohibited(position, t)` → `bool`
Inside an obstacle or active geofence.

`prohibited_regions_at(position, t)` → `list`
Restrictions that contain the position.

`Box3D, Cylinder3D, Geometry` geometries

`Box3D(minimum_x, maximum_x, minimum_y, maximum_y, minimum_z, maximum_z)`
`Cylinder3D(center_x, center_y, radius, minimum_z, maximum_z)`
Axis-aligned box and vertical cylinder [m]. Both implement `contains(position)` → `bool`. A custom shape inherits `Geometry` and implements `contains` (the built-in detour planner supports only boxes and cylinders).

`Obstacle` dataclass

`Obstacle(obstacle_id, geometry, name=None)`
Permanently prohibited region; entering it is a collision.

`Geofence` dataclass

`Geofence(geofence_id, geometry, active_from=None, active_until=None, name=None)`
KEEP_OUT region; entering it is a violation. `active_from` and `active_until` give an optional validity interval; `is_active_at(t)` and `prohibits(position, t)` test it.

`WindZone` dataclass

`WindZone(wind_id, geometry, velocity, active_from=None, active_until=None, name=None)`
Region where the constant `velocity` vector [m/s] is added to the motion of the UASs. Overlapping zones add up.

6.5 `musas.algorithms`: built-in algorithms

The built-in algorithms are simple reference implementations. They are meant to exercise the framework and to serve as baselines, not as definitive UTM algorithms. The contracts they implement are described in Chapter 7.

6.5.1 Vehicle models

`FirstOrderDynamics` — DynamicsAlgorithm

`FirstOrderDynamics(max_acceleration=3.0, velocity_time_constant=1.0)`
The velocity follows the command with time constant `velocity_time_constant` [s] (0 = instantaneous), limited by `max_acceleration` [m/s^2]. Wind is added to the resulting motion.

`WaypointGuidance` — GuidanceAlgorithm

`WaypointGuidance(min_speed=0.0, max_speed=20.0)`
Commands the velocity that reaches the target waypoint at its scheduled time, bounded by the speed limits [m/s].

`PerfectSensor, GaussianSensor` — SensorModel

`PerfectSensor()`
`GaussianSensor(position_standard_deviation=1.0, velocity_standard_deviation=0.1, seed=None)`
Perfect measurements, or measurements with independent zero-mean Gaussian noise.

`ConstantTimeProcessing` — ProcessingTimeModel

`ConstantTimeProcessing(delay=0.0)`
Same processing delay [s] for every operation.

6.5.2 Strategic layer

`MinimumSeparationConflictPrediction` — ConflictPredictionAlgorithm

`MinimumSeparationConflictPrediction(minimum_separation_distance=30.0, sampling_interval=0.5, terminal_buffer=1.0, terminal_radius=5.0)`
Samples the proposal and each approved plan over their common time window and reports the first traffic conflict with each UAS (distance below the separation) and the first airspace conflict with each restriction. A terminal rule also reports a plan that departs less than `terminal_radius` from the destination of another, at most `terminal_buffer` seconds after its arrival.

`RecursiveTimeShiftDeconfliction` — StrategicDeconflictionAlgorithm

`RecursiveTimeShiftDeconfliction(conflict_prediction_algorithm=None, path_planner=None, time_shift_step=5.0, maximum_time_shifts=20)`
Removes airspace conflicts with `path_planner` and then delays the departure in steps of `time_shift_step` until the plan is separated from every approved plan (first-come, first-served; approved plans are never changed). Rejects the proposal after `maximum_time_shifts`.

`DetourPathPlanner` — helper

`DetourPathPlanner(conflict_prediction_algorithm=None, lateral_clearance=10.0, vertical_clearance=10.0, maximum_maneuvers=4, allow_lateral=True, allow_vertical=True)`

Plans a short detour around obstacles and active geofences: lateral first, vertical as fallback. `plan(flight_plan, airspace)` returns the new plan or `None`.

6.5.3 Tactical layer

`FlightPlanAwareTracking` — TrackingAlgorithm

`FlightPlanAwareTracking()`
Predicts along the authorized plan, corrected by the observed position offset and velocity error, so disturbances such as wind are carried into the track. The track ends at the predicted arrival.

`MinimumSeparationTacticalConflictPrediction` — TacticalConflictPredictionAlgorithm

`MinimumSeparationTacticalConflictPrediction(minimum_separation_distance=30.0, maximum_time_difference=0.5)`
Compares each pair of tracks at matching instants (within `maximum_time_difference`) and checks every track against obstacles and active geofences.

`HybridGeometricManeuver` — TacticalDeconflictionAlgorithm

`HybridGeometricManeuver(conflict_prediction_algorithm=None, minimum_separation_distance=30.0, vertical_offset=40.0, airspace_clearance=10.0, maneuver_duration=5.0, recovery_duration=8.0, minimum_speed=1.0, maximum_speed=25.0, cooldown=5.0, maximum_escape_distance=500.0)`
Vertical offset for traffic conflicts and lateral detour (or escape, when the UAS is already inside) for airspace conflicts. The lower-priority UAS maneuvers (the later one in FCFS order for equal priorities). Candidates are validated against the other tracks and restrictions, and `cooldown` limits successive maneuvers of the same UAS.

`ExtendedHybridGeometricManeuver` — TacticalDeconflictionAlgorithm

`ExtendedHybridGeometricManeuver(..., vertical_offsets=(40.0, 60.0, 80.0), lateral_offsets=(45.0, 70.0), desired_separation_factor=1.5, emergency_time_to_conflict=6.0, ...)`
Extended variant for disturbed traffic: evaluates several vertical and lateral candidates, targets `desired_separation_factor` times the separation, keeps the observed drift in the candidate predictions, and accepts only a candidate that improves the predicted separation. The other parameters are those of `HybridGeometricManeuver`.

6.5.4 Built-in conflict types

Class	Fields
`StrategicTrafficConflict`	`timestamp`, `conflicting_uav`, `proposed_position`, `conflicting_position`, `separation_distance`
`StrategicAirspaceConflict`	`timestamp`, `position`, `restriction`
`TacticalTrafficConflict`	`timestamp`, `uav_name`, `other_uav`, `position`, `other_position`, `separation_distance`
`TacticalAirspaceConflict`	`timestamp`, `uav_name`, `position`, `restriction`

7

Writing Your Own Algorithms

Each contract is an abstract base class with a single abstract method. To create an algorithm, inherit the contract, implement the method, and pass an instance to `Simulation` or `add_uav`. All contracts are exported by `musas.algorithms`.

Contract	Method	Called
`ConflictPredictionAlgorithm`	`predict`	For each proposal
`StrategicDeconflictionAlgorithm`	`deconflict`	After the strategic prediction
`TrackingAlgorithm`	`predict`	For each report
`TacticalConflictPredictionAlgorithm`	`predict`	Every tactical assessment
`TacticalDeconflictionAlgorithm`	`deconflict`	When the assessment finds conflicts
`GuidanceAlgorithm`	`calculate_commanded_velocity`	Every flight update
`DynamicsAlgorithm`	`step`	Every flight update
`SensorModel`	`measure`	For each report
`ProcessingTimeModel`	`calculate`	Before each module operation

7.1 Strategic contracts

`ConflictPredictionAlgorithm.predict` abstract

`predict(proposed_flight_plan, approved_flight_plans, airspace) → list[Conflict]`
Compares a proposal with every approved plan (`dict` by UAS name) and with the airspace snapshot. Return an empty list when there is no conflict.

`StrategicDeconflictionAlgorithm.deconflict` abstract

`deconflict(proposed_flight_plan, conflicts, inactive_flight_plans, active_flight_plans, airspace) → (StrategicDecision, FlightPlan | None, list[FlightPlanUpdate])`
Returns the decision, the counter-proposal (only with `COUNTER_PROPOSAL`), and updates to other plans. Updates may change only *inactive* plans (approved but not departed); active plans are read only context.

7.2 Tactical contracts

`TrackingAlgorithm.predict` abstract

`predict(uav_name, report_history, flight_plan, prediction_start, prediction_horizon, prediction_step) → TrackPrediction`
Receives the last `history_window` reports and the active plan of one UAS; returns the estimated state at `prediction_start` and `prediction_horizon` future states spaced by `prediction_step`.

`TacticalConflictPredictionAlgorithm.predict` abstract

`predict(tracks, airspace) → list[Conflict]`
Receives the current track of every active UAS and the airspace snapshot; each pair of UASs should be evaluated once.

`TacticalDeconflictionAlgorithm.deconflict` abstract

`deconflict(conflicts, tracks, flight_plans, airspace) → list[FlightPlanUpdate]`
Returns replacement plans for the UASs that must maneuver. A tactical plan usually starts at the current estimated state and rejoins the original route.

7.3 Vehicle contracts

`GuidanceAlgorithm.calculate_commanded_velocity` abstract

`calculate_commanded_velocity(current_position, current_time, target_waypoint) → list[float]`

`DynamicsAlgorithm.step` abstract

`step(position, velocity, commanded_velocity, dt) → (new_position, new_velocity)`
Advances the state by `dt`; wind is added by the simulation.

`SensorModel.measure` abstract

`measure(position, velocity) → {"position": [...], "velocity": [...]}`

7.4 Rules for every algorithm

- Do not modify the objects you receive; build new `FlightPlans`.
- Keep the `priority` of the operation in every new plan; the framework rejects updates that change it.
- Return at most one `FlightPlanUpdate` per UAS in each call.
- Use only what you receive: wind is never given to the algorithms, and geofences appear only after their activation.
- Keep state in the instance if you need it (for example, a cooldown), and create a new instance for each simulation.

7.5 Template: a tactical algorithm

```python
from musas.algorithms import (TacticalConflictPredictionAlgorithm,
                              TacticalDeconflictionAlgorithm,
                              TacticalTrafficConflict)
from musas.domain import FlightPlan, FlightPlanUpdate, Waypoint


class ClimbOnConflict(TacticalDeconflictionAlgorithm):
    """The second UAS of each traffic conflict climbs 40 m for 10 s."""

    def deconflict(self, conflicts, tracks, flight_plans, airspace):
        updates = {}
        for conflict in conflicts:
            if not isinstance(conflict, TacticalTrafficConflict):
                continue
            name = conflict.other_uav
            if name in updates:
                continue
            state = tracks[name].estimated_state
            plan = flight_plans[name]
            x, y, z = state.position
            t = state.timestamp
            remaining = [w for w in plan.waypoint_list if w.t > t + 10.0]
            updates[name] = FlightPlanUpdate(name, FlightPlan(
                [Waypoint(x, y, z, t), Waypoint(x, y, z + 40.0, t + 5.0),
                 *remaining],
                priority=plan.priority,
            ))
        return list(updates.values())
```

8

Troubleshooting

Message or symptom	Cause and solution
`ModuleNotFoundError: musas`	Install the package in the active environment (Chapter 2); in Jupyter, restart the kernel after `%pip install`.
`both strategic algorithms must be provided together`	Pass both strategic algorithms, or neither.
`tracking_algorithm is required for tactical deconfliction`	Add a tracking algorithm when using the tactical layer.
`until must be greater than the current time`	`run(until)` takes an absolute time; use a value larger than the previous one.
`simulation is closed`	A closed simulation cannot continue; create a new one.
`simulation dataset already exists`	Pass `overwrite_dataset=True` or choose another `dataset_path`.
The plot does not show a new obstacle	Figures are rebuilt from the dataset; run the simulation again after changing the scenario.
No tactical reaction	The conflict is beyond the prediction horizon (`prediction_horizon` × `prediction_step`) or the cooldown is active.
MP4 export fails	Install `ffmpeg`, or save as `.gif`.